

Università degli Studi di Bari Aldo Moro

Corso di Laurea in Informatica

Metodi Avanzati di Programmazione

A.A. 2025/2026

Regression Tree Miner

Manuale utente e guida all'installazione

Progetto base

Studente: Giovane Francesco Alessandro

Matricola: 799104

Email: f.giovane@studenti.uniba.it

Indice

1	Introduzione	1
1.1	Scopo del progetto	1
1.2	Requisiti di sistema	2
1.3	Contenuto della distribuzione	2
2	Architettura del sistema	3
2.1	Panoramica	3
2.2	Package <code>client</code>	4
2.3	Package <code>utility</code>	4
2.4	Package <code>server</code>	5
2.5	Package <code>data</code>	5
2.6	Package <code>database</code>	6
2.7	Package <code>tree</code>	6
3	Installazione e avvio	7
3.1	Preparazione	7
3.2	Avvio di MySQL	7
3.2.1	Controllo del servizio su Linux	8
3.2.2	Controllo del servizio su Windows	8
3.2.3	Creazione automatica del database	8
3.2.4	Verifica della configurazione	10
3.3	Avvio del server	10
3.3.1	Avvio del server su Linux	10
3.3.2	Avvio del server su Windows	11
3.4	Avvio del client	11
3.4.1	Avvio del client su Linux	11

3.4.2	Avvio del client su Windows	12
3.4.3	Connessione a un server remoto	13
4	Utilizzo del client	14
4.1	Menu principale	14
4.2	Apprendimento di un nuovo albero	14
4.3	Predizione guidata	15
4.4	Caricamento di un albero salvato	15
4.5	Esempio completo di sessione	16
5	Chiusura dell'applicazione	18
5.1	Chiusura dei client	18
5.2	Arresto del server	18
5.3	Conservazione degli alberi	18

Elenco delle figure

2.1	Relazioni tra i package del progetto base.	3
2.2	Diagramma delle classi del package <code>client</code>	4
2.3	Diagramma delle classi del package <code>utility</code>	4
2.4	Diagramma delle classi del package <code>server</code>	5
2.5	Diagramma delle classi del package <code>data</code>	5
2.6	Diagramma delle classi del package <code>database</code>	6
2.7	Diagramma delle classi del package <code>tree</code>	6
3.1	Avvio del server tramite script su Linux.	11
3.2	Menu del client avviato tramite script su Linux.	12
4.1	Apprendimento di un albero dalla tabella <code>provaC</code>	15
4.2	Esempio di predizione guidata e visualizzazione della classe predetta. . .	15
4.3	Caricamento di un albero serializzato e avvio di una nuova predizione. . .	16
4.4	Sessione di apprendimento e predizione completa.	16
4.5	Sessione di caricamento e predizione completa.	17

Introduzione

1.1 Scopo del progetto

Regression Tree Miner è un'applicazione Java client-server finalizzata all'apprendimento e all'utilizzo di alberi di regressione a partire da dati memorizzati in un database MySQL.

Un albero di regressione è un modello predittivo che suddivide progressivamente gli esempi del training set in gruppi più omogenei, sulla base dei valori degli attributi esplicativi. Ogni nodo interno rappresenta una condizione applicata a un attributo discreto o continuo, mentre ogni nodo foglia contiene il valore numerico utilizzato come predizione.

Durante la fase di apprendimento, il sistema acquisisce gli esempi da una tabella del database e ricerca, per ciascun nodo, la suddivisione che riduce maggiormente l'errore quadratico degli esempi coperti. Il processo continua ricorsivamente fino alla costruzione completa dell'albero.

Il progetto permette all'utente di:

- acquisire un training set da una tabella MySQL;
- costruire un albero di regressione;
- salvare l'albero appreso in un archivio serializzato con estensione `.dmp`;
- caricare un albero precedentemente salvato;
- effettuare predizioni guidate tramite un client a riga di comando;
- servire più client mediante thread distinti sul server.

L'applicazione è organizzata secondo un'architettura client-server. Il client fornisce l'interfaccia testuale attraverso la quale l'utente seleziona le operazioni e inserisce i dati richiesti. Il server gestisce l'accesso al database, l'apprendimento, il salvataggio e il caricamento degli alberi, oltre all'esecuzione delle predizioni.

La comunicazione tra client e server avviene tramite socket TCP e consente di separare l'interfaccia utente dalla logica applicativa e dalla gestione dei dati. Il server è inoltre in grado di gestire più connessioni mediante thread distinti.

1.2 Requisiti di sistema

Per eseguire il progetto sono necessari:

- un sistema operativo in grado di eseguire Java e MySQL;
- Java Runtime Environment o Java Development Kit 21 o superiore;
- MySQL 8 o compatibile;
- un terminale;
- una porta TCP libera per il server, impostata nel progetto al valore 8080.

Per verificare la presenza delle versioni corrette dei programmi richiesti è possibile aprire un terminale ed eseguire:

```
java -version
mysql --version
```

Se uno dei due comandi non viene riconosciuto, installare il componente mancante prima di procedere.

Su Linux Mint è possibile installare i pacchetti principali con:

```
sudo apt update
sudo apt install openjdk-21-jdk mysql-server
```

1.3 Contenuto della distribuzione

La struttura logica del progetto base è la seguente:

```
progetto_base/
|-- src/
|   |-- client/
|   |-- server/
|   '-- setup.sql
|-- dist/
|   |-- avvia_client.bat
|   |-- avvia_server.bat
|   |-- avvia_client.sh
|   |-- avvia_server.sh
|   |-- client.jar
|   '-- server.jar
|-- javadoc/
|   |-- javadocForClient/
|   '-- javadocForServer/
|-- uml/
|-- manuale_utente_progetto_base.pdf
'-- casi_di_test_progetto_base.pdf
```

Architettura del sistema

2.1 Panoramica

Regression Tree Miner adotta un'architettura client-server:

1. il client stabilisce una connessione socket verso il server;
2. il server riceve le richieste e assegna ogni client a un thread dedicato;
3. per l'apprendimento, il server acquisisce i dati dal database MySQL tramite JDBC;
4. il package `tree` costruisce o carica l'albero di regressione;
5. durante la predizione, il server invia al client una sequenza di domande;
6. il client restituisce la scelta effettuata dall'utente;
7. al raggiungimento di una foglia, il server comunica il valore numerico predetto.

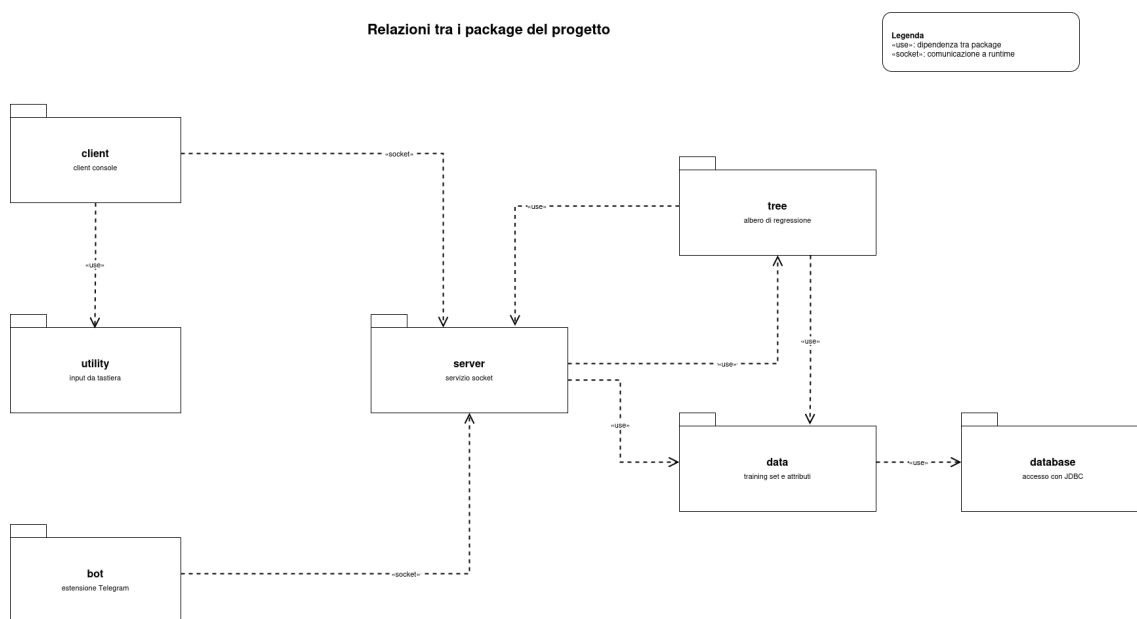


Figura 2.1: Relazioni tra i package del progetto base.

2.2 Package client

Il package `client` contiene il punto di avvio del client a riga di comando. La classe principale interpreta i parametri di connessione, mostra il menu e comunica con il server tramite stream di oggetti.

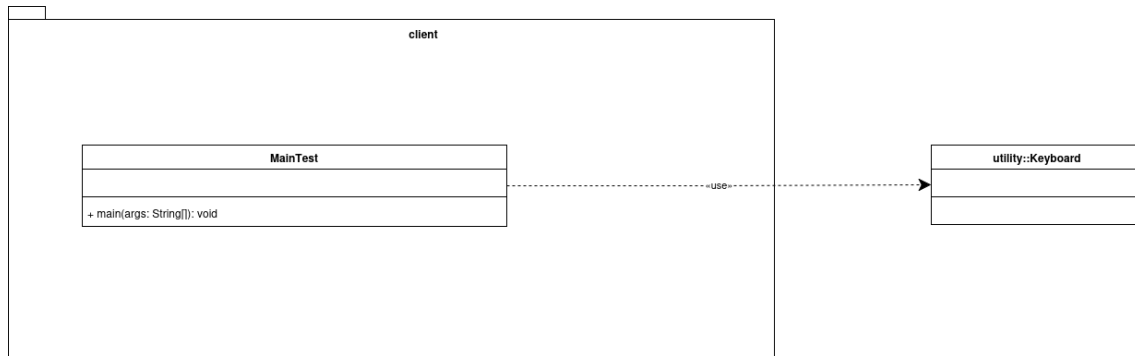


Figura 2.2: Diagramma delle classi del package `client`.

2.3 Package utility

Il package `utility` contiene la classe `Keyboard`, utilizzata per leggere e validare i valori inseriti dall'utente.

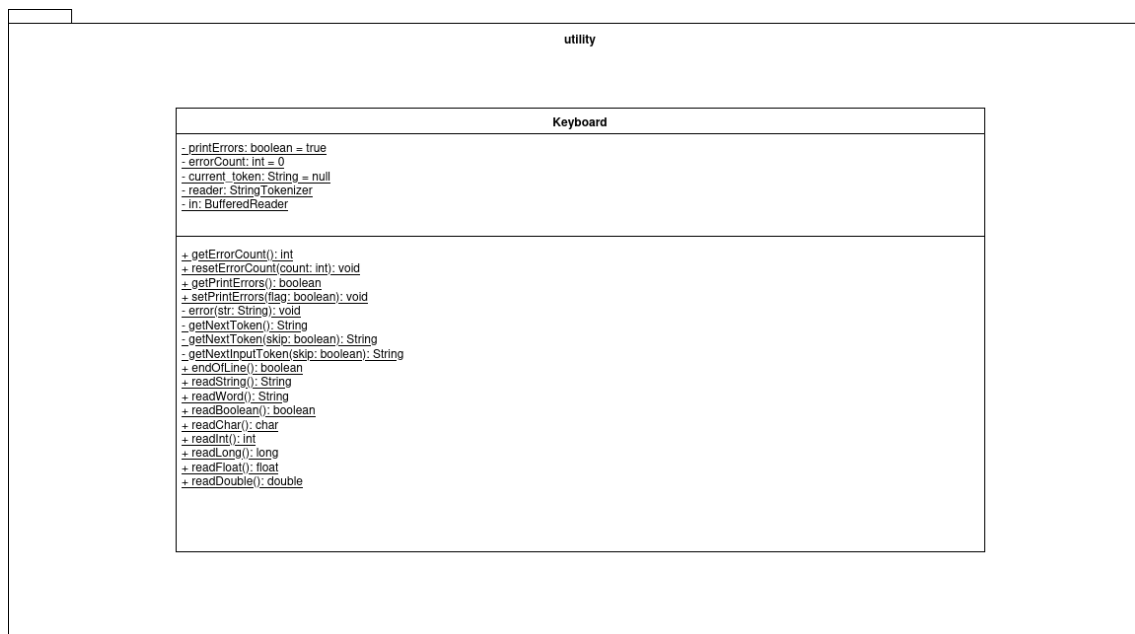


Figura 2.3: Diagramma delle classi del package `utility`.

2.4 Package server

Il package `server` gestisce la porta di ascolto, crea un thread per ogni connessione e coordina acquisizione del training set, apprendimento, caricamento e predizione.

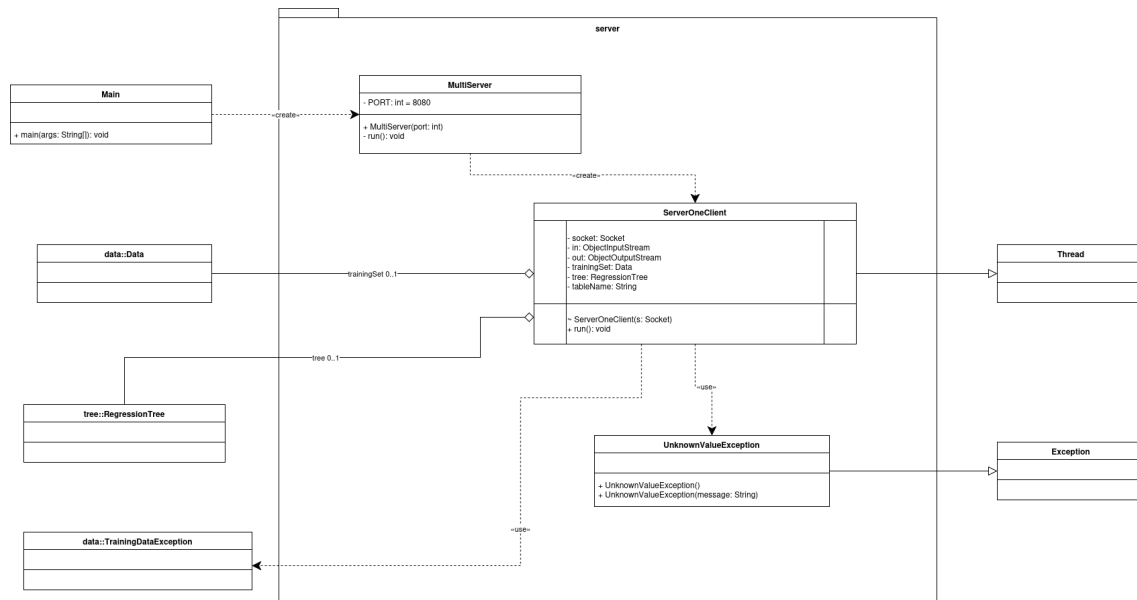


Figura 2.4: Diagramma delle classi del package `server`.

2.5 Package data

Il package `data` rappresenta gli esempi e gli attributi del training set. L'ultima colonna della tabella è interpretata come attributo target e deve essere numerica.

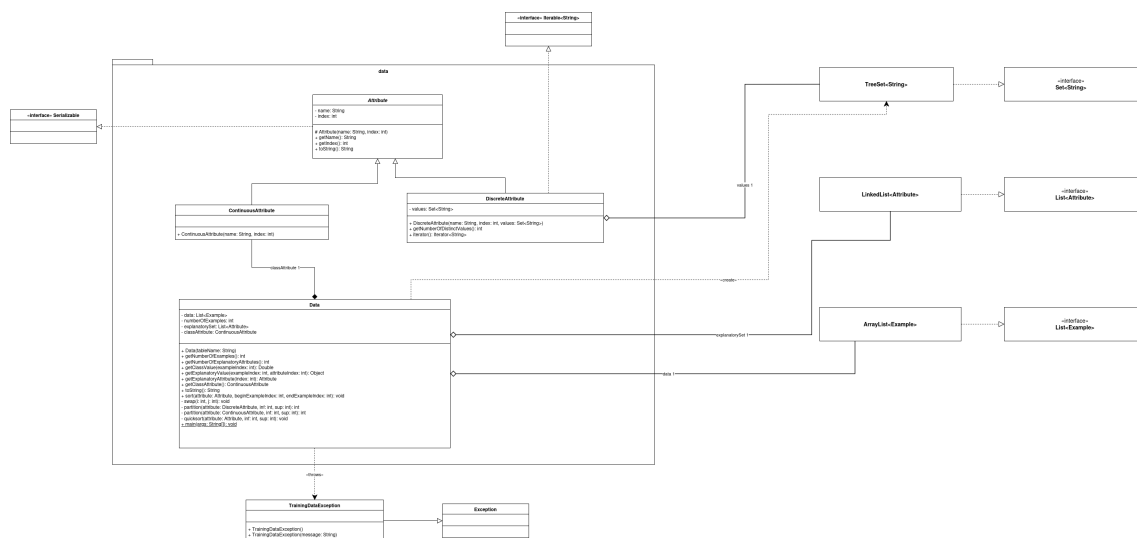


Figura 2.5: Diagramma delle classi del package `data`.

2.6 Package database

Il package `database` realizza l'accesso al DBMS tramite JDBC, legge lo schema della tabella, acquisisce le transazioni e individua i valori distinti degli attributi.

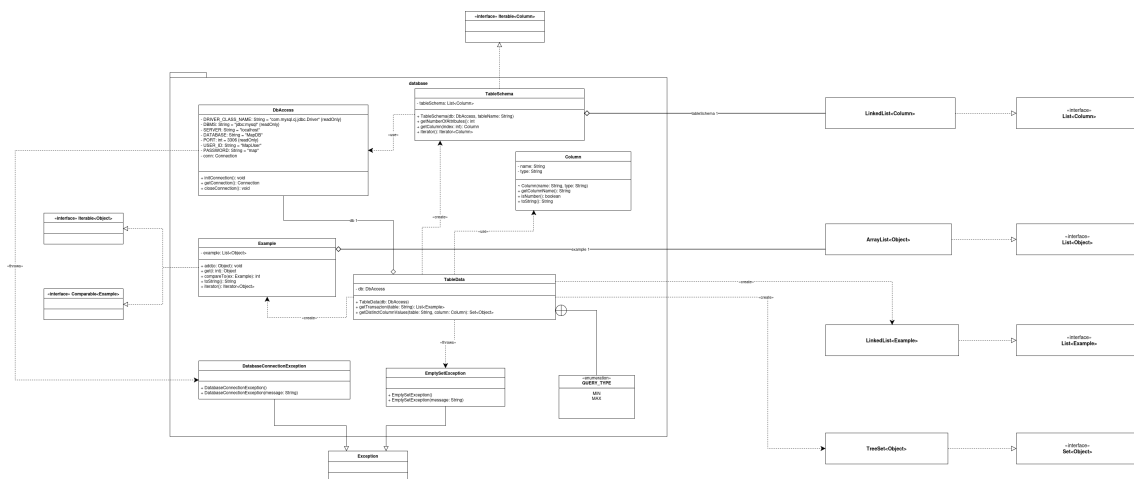


Figura 2.6: Diagramma delle classi del package `database`.

2.7 Package tree

Il package `tree` modella i nodi dell'albero di regressione e contiene le operazioni di apprendimento, serializzazione, caricamento e predizione.

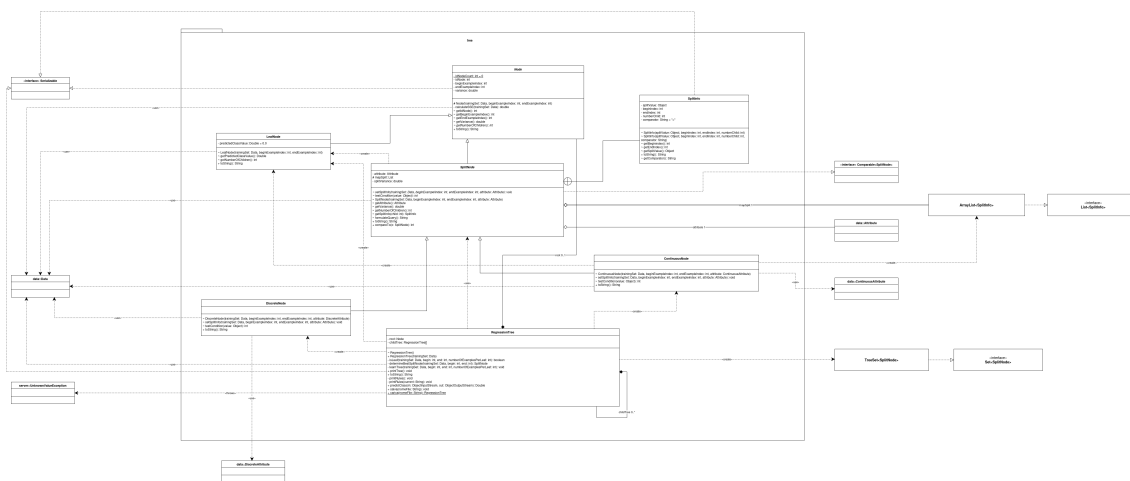


Figura 2.7: Diagramma delle classi del package `tree`.

Installazione e avvio

3.1 Preparazione

Estrarre il file .zip consegnato. Aprire la cartella `progetto_base` all'interno di un terminale.

Nella cartella `/dist` saranno presenti tutti i file necessari all'avvio del programma.

L'ordine corretto di avvio è:

1. MySQL;
2. server;
3. client.

3.2 Avvio di MySQL

Prima di avviare il server è necessario verificare che il servizio MySQL sia attivo e che sia disponibile un database compatibile con il progetto.

Se nel sistema sono già presenti il database, l'utente e le tabelle richieste dal progetto, è sufficiente controllare che MySQL sia in esecuzione e procedere con la sezione successiva.

La configurazione utilizzata dal progetto è la seguente:

Parametro	Valore
Database	MapDB
Utente	MapUser
Password	map
Host	localhost
Porta MySQL	3306

Il database deve inoltre contenere almeno una tabella idonea all'apprendimento di un albero di regressione. L'ultima colonna della tabella deve rappresentare l'attributo target e deve essere di tipo numerico.

3.2.1 Controllo del servizio su Linux

Su Linux Mint e sui sistemi che utilizzano `systemd`, lo stato del servizio può essere controllato con:

```
sudo systemctl status mysql
```

Se MySQL non è attivo, avviarlo con:

```
sudo systemctl start mysql
```

3.2.2 Controllo del servizio su Windows

Su Windows il nome del servizio può variare in base alla versione e al metodo utilizzato per installare MySQL. Un nome comune è `MySQL80`.

Aprire PowerShell come amministratore e cercare i servizi MySQL disponibili:

```
Get-Service -Name "MySQL*"
```

Se, ad esempio, il servizio si chiama `MySQL80`, può essere avviato con:

```
Start-Service -Name MySQL80
```

In alternativa, dal Prompt dei comandi avviato come amministratore:

```
sc query MySQL80  
net start MySQL80
```

Se il servizio presenta un nome differente, sostituire `MySQL80` con quello mostrato dal sistema.

3.2.3 Creazione automatica del database

Se la configurazione richiesta non è già presente, è possibile utilizzare lo script:

```
src/setup.sql
```

Il percorso è relativo alla cartella principale del progetto.

Lo script:

- crea il database `MapDB`, se non esiste;
- ricrea l'utente `MapUser` con password `map`;
- assegna all'utente i privilegi necessari su `MapDB`;
- crea e popola le tabelle `prova`, `provaC` e `servo`.

Nota importante. Lo script elimina e ricrea l'utente `MapUser` e le tabelle `prova`, `provaC` e `servo`. Eventuali dati già presenti in queste tabelle verranno sostituiti.

Esecuzione dello script su Linux

Aprire un terminale nella cartella principale del progetto ed eseguire:

```
sudo mysql < src/setup.sql
```

Esecuzione dello script su Windows

Su Windows, la procedura consiste nell'aprire il client MySQL e utilizzare il comando `SOURCE`.

Dal menu Start, avviare *MySQL Command Line Client*. In alternativa, aprire PowerShell o il Prompt dei comandi ed eseguire:

```
mysql -u root -p
```

Quando richiesto, inserire la password dell'utente amministratore `root`.

Dopo l'accesso al client MySQL, eseguire lo script mediante il comando `SOURCE`, indicando il percorso completo del file `src/setup.sql`.

Nel percorso è consigliabile utilizzare le barre oblique:

```
SOURCE C:/percorso/completo/del/progetto/src/setup.sql;
```

Ad esempio:

```
SOURCE C:/Users/NomeUtente/Desktop/progetto_base/src/setup.sql;
```

Il comando deve terminare con il punto e virgola.

Se il comando `mysql` non viene riconosciuto dal terminale, è possibile:

- avviare *MySQL Command Line Client* direttamente dal menu Start;
- oppure eseguire `mysql.exe` indicando il percorso completo.

Un possibile percorso, che può variare in base alla versione installata, è:

```
"C:\Program Files\MySQL\MySQL Server 8.0\bin\mysql.exe" -u root -p
```

Al termine dell'esecuzione dello script, uscire dal client MySQL con:

```
EXIT;
```

Nota importante. Si fa presente che le precedenti indicazioni riguardanti il setup del database su Windows potrebbero essere inesatte. Non è stato possibile testare il progetto su un Sistema Operativo Windows.

3.2.4 Verifica della configurazione

Per verificare che il database e le tabelle siano stati creati correttamente, accedere con l'utente del progetto:

```
mysql -u MapUser -p -h localhost MapDB
```

Quando richiesto, inserire la password:

```
map
```

Mostrare quindi le tabelle disponibili:

```
SHOW TABLES;
```

Il risultato deve includere:

```
prova  
provaC  
servo
```

È possibile verificare il contenuto di una tabella, ad esempio **provaC**, con:

```
SELECT * FROM provaC;
```

Dopo aver verificato che MySQL sia attivo e che il database sia correttamente configurato, è possibile procedere con l'avvio del server.

3.3 Avvio del server

Prima di avviare il server, verificare che il servizio MySQL sia attivo e che il database richiesto dal progetto sia correttamente configurato.

3.3.1 Avvio del server su Linux

Aprire un terminale nella cartella principale del progetto.

Alla prima esecuzione potrebbe essere necessario rendere eseguibile lo script:

```
chmod +x dist/avvia_server.sh
```

Avviare quindi il server con:

```
./dist/avvia_server.sh
```

In alternativa, lo script può essere eseguito esplicitamente tramite Bash:

```
bash dist/avvia_server.sh
```

Lo script si sposta automaticamente nella cartella **dist** ed esegue:

```
java -jar server.jar
```

Il server resta in ascolto sulla porta TCP 8080. Il terminale deve rimanere aperto durante l'utilizzo del client.



```
giovane@giovanePC:~/Documenti/Università/Secondo_Anno/2_Semestre/MAP/Progetto/RegressionTreeMiner/progetto_esteso$ ./dist/avvia_server.sh
Server avviato sulla porta 8080
```

Figura 3.1: Avvio del server tramite script su Linux.

3.3.2 Avvio del server su Windows

Aprire il Prompt dei comandi o PowerShell nella cartella principale del progetto ed eseguire:

```
dist\avvia_server.bat
```

È anche possibile aprire la cartella **dist** tramite Esplora file e fare doppio clic sul file:

```
avvia_server.bat
```

Lo script si sposta automaticamente nella cartella in cui si trova ed esegue:

```
java -jar server.jar
```

Al termine dell'esecuzione, il comando **pause** mantiene aperta la finestra, permettendo di leggere eventuali messaggi o errori.

Nota importante. Non è stato possibile verificare direttamente la correttezza dello script `avvia_server.bat` su un sistema Windows.

3.4 Avvio del client

Il client deve essere avviato soltanto dopo il server. Lo script fornito configura automaticamente la connessione al server locale utilizzando l'indirizzo `127.0.0.1` e la porta `8080`.

3.4.1 Avvio del client su Linux

Aprire un secondo terminale nella cartella principale del progetto.

Alla prima esecuzione potrebbe essere necessario rendere eseguibile lo script:

```
chmod +x dist/avvia_client.sh
```

Avviare quindi il client con:

```
./dist/avvia_client.sh
```

In alternativa:

```
bash dist/avvia_client.sh
```

Lo script si sposta automaticamente nella cartella **dist** ed esegue:

```
java -jar client.jar 127.0.0.1 8080
```

I parametri utilizzati indicano:

127.0.0.1 indirizzo del server eseguito sulla stessa macchina;

8080 porta TCP sulla quale il server resta in ascolto.



```
giovane@giovanePC:~/Documenti/Università/Secondo_Anno/2_Semestre/MAP/Progetto/RegressionTreeMiner/progetto_base$ ./dist/avvia_client.sh
Socket[addr=/127.0.0.1,port=8080,localport=57586]

==== Regression Tree Miner ====
1) Apprendi un nuovo albero da una tabella del database
2) Carica un albero già salvato da archivio
Scelta: █
```

Figura 3.2: Menu del client avviato tramite script su Linux.

3.4.2 Avvio del client su Windows

Aprire un secondo Prompt dei comandi o una seconda finestra di PowerShell nella cartella principale del progetto ed eseguire:

```
dist\avvia_client.bat
```

In alternativa, aprire la cartella **dist** tramite Esplora file e fare doppio clic sul file:

```
avvia_client.bat
```

Lo script esegue automaticamente:

```
java -jar client.jar 127.0.0.1 8080
```

Anche in questo caso, il comando **pause** mantiene aperta la finestra dopo la conclusione del programma, consentendo di leggere eventuali messaggi.

Nota importante. Non è stato possibile verificare direttamente la correttezza dello script `avvia_client.bat` su un sistema Windows. Le immagini mostrate nel manuale si riferiscono quindi all'esecuzione su Linux.

3.4.3 Connessione a un server remoto

Gli script forniti avviano il client collegandolo al server locale 127.0.0.1:8080.

Per collegarsi a un server eseguito su un altro computer è necessario avviare manualmente il JAR, sostituendo l'indirizzo IP:

```
java -jar dist/client.jar 192.168.1.50 8080
```

L'indirizzo indicato deve essere sostituito con quello effettivo del computer sul quale è in esecuzione il server.

Utilizzo del client

4.1 Menu principale

Il client propone due operazioni principali:

1. apprendere un nuovo albero da una tabella del database;
2. caricare un albero già salvato da archivio.

L'utente deve inserire il numero corrispondente all'operazione desiderata.

4.2 Apprendimento di un nuovo albero

Per apprendere un albero:

1. selezionare l'opzione 1;
2. inserire il nome della tabella, ad esempio `provaC`;
3. attendere l'acquisizione dei dati e l'apprendimento;
4. verificare il messaggio di completamento;
5. proseguire con la predizione guidata.

L'albero viene salvato dal server nella directory corrente con un nome derivato dalla tabella, ad esempio:

```
provaC.dmp
```

```
==== Regression Tree Miner ====
1) Apprendi un nuovo albero da una tabella del database
2) Carica un albero già salvato da archivio
Scelta: 1

Inserisci il nome della tabella del database:
provaC

[1/3] Caricamento del training set dal database...
[2/3] Apprendimento dell'albero di regressione...
[3/3] Albero appreso e salvato.

--- Predizione guidata ---
Scegli il ramo digitando il numero corrispondente:
0:X=A
1:X=B
Scelta: █
```

Figura 4.1: Apprendimento di un albero dalla tabella `provaC`.

4.3 Predizione guidata

Durante la predizione il server invia una domanda relativa a un nodo dell'albero. Il client mostra i rami disponibili e l'utente inserisce il numero del ramo corrispondente al valore del nuovo esempio.

Il procedimento continua fino al raggiungimento di una foglia, quando viene mostrata la classe numerica predetta.

```
--- Predizione guidata ---
Scegli il ramo digitando il numero corrispondente:
0:X=A
1:X=B

Scelta: 0
0:Y<=2.0
1:Y>2.0

Scelta: 1
Classe predetta: 1.5

Vuoi effettuare un'altra predizione? (s/n)
█
```

Figura 4.2: Esempio di predizione guidata e visualizzazione della classe predetta.

4.4 Caricamento di un albero salvato

Per riutilizzare un albero già serializzato:

1. verificare che il file `.dmp` si trovi nella directory di lavoro del server;
2. l'opzione 2;
3. inserire il nome dell'archivio richiesto dal programma;
4. attendere il caricamento;
5. effettuare la nuova predizione guidata.

```
==== Regression Tree Miner ====
1) Apprendi un nuovo albero da una tabella del database
2) Carica un albero già salvato da archivio
Scelta: 2

Inserisci il nome dell'archivio senza estensione .dmp:
provaC

Caricamento dell'albero di regressione dall'archivio...
Albero caricato correttamente.

--- Predizione guidata ---
Scegli il ramo digitando il numero corrispondente:
0:X=A
1:X=B

Scelta: 1
Classe predetta: 10.0

Vuoi effettuare un'altra predizione? (s/n)
```

Figura 4.3: Caricamento di un albero serializzato e avvio di una nuova predizione.

4.5 Esempio completo di sessione

```
giovane@giovanePC:~/Documenti/Università/Secondo_Anno/2_Semestre/MAP/Progetto/RegressionTreeMiner/progetto_base$ ./dist/avvia_client.sh
Socket[addr=/127.0.0.1,port=8080,localport=56224]

==== Regression Tree Miner ====
1) Apprendi un nuovo albero da una tabella del database
2) Carica un albero già salvato da archivio
Scelta: 1

Inserisci il nome della tabella del database:
servo

[1/3] Caricamento del training set dal database...
[2/3] Apprendimento dell'albero di regressione...
[3/3] Albero appreso e salvato.

--- Predizione guidata ---
Scegli il ramo digitando il numero corrispondente:
0:pgain=3
1:pgain=4
2:pgain=5
3:pgain=6

Scelta: 2
0:screw=A
1:screw=B
2:screw=C
3:screw=D
4:screw=E

Scelta: 4
Classe predetta: 0.431252

Vuoi effettuare un'altra predizione? (s/n)
n
giovane@giovanePC:~/Documenti/Università/Secondo_Anno/2_Semestre/MAP/Progetto/RegressionTreeMiner/progetto_base$
```

Figura 4.4: Sessione di apprendimento e predizione completa.

```
giovane@giovanePC:~/Documenti/Università/Secondo_Anno/2_Semestre/MAP/Progetto/RegressionTreeMiner/progetto_base$ ./dist/avvia_client.sh
Socket[addr=/127.0.0.1,port=8080,localport=52226]

==== Regression Tree Miner ====
1) Apprendi un nuovo albero da una tabella del database
2) Carica un albero già salvato da archivio
Scelta: 2

Inserisci il nome dell'archivio senza estensione .dmp:
servo

Caricamento dell'albero di regressione dall'archivio...
Albero caricato correttamente.

--- Predizione guidata ---
Scegli il ramo digitando il numero corrispondente:
0:pgain=3
1:pgain=4
2:pgain=5
3:pgain=6

Scelta: 3
0:screw=A
1:screw=B
2:screw=C
3:screw=D
4:screw=E

Scelta: 2
Classe predetta: 0.4687525

Vuoi effettuare un'altra predizione? (s/n)
n
giovane@giovanePC:~/Documenti/Università/Secondo_Anno/2_Semestre/MAP/Progetto/RegressionTreeMiner/progetto_base$
```

Figura 4.5: Sessione di caricamento e predizione completa.

L'output esatto dipende dai valori inseriti e dal percorso seguito nell'albero.

Chiusura dell'applicazione

5.1 Chiusura dei client

Nei terminali dei client premere:

```
CTRL+C
```

5.2 Arresto del server

Nel terminale del server premere:

```
CTRL+C
```

Arrestare il server solo dopo aver chiuso i client collegati.

5.3 Conservazione degli alberi

I file `.dmp` prodotti dal server devono essere conservati se si desidera riutilizzare gli alberi senza ripetere l'apprendimento. Prima di spostare il progetto, verificare la directory dalla quale è stato avviato `server.jar`, perché gli archivi vengono creati rispetto alla directory di lavoro.